



Move Semantics, RAII & the Rule of Five

Elias Farhan



What are we doing today?

- Why C++ needs **RAII**
- What **move semantics** actually is (and isn't)
- The **Rule of Five** (and zero, and three)
- How to write classes the compiler, and the STL, can use efficiently
- A checklist of common mistakes

Source: *Notebook C++: About Move Semantics*, Andreas Fertig (2024).



A class that owns memory

The destructor `delete[]`s `data_`. The constructor allocates. What happens when we **copy** this object?

```
class String {  
public:  
    String(const char* s);  
    ~String();  
private:  
    char* data_;  
    size_t len_;  
};
```



What if there was a way...

to tie the lifetime of a heap allocation to the lifetime of a stack object, so cleanup is **automatic**, even on exceptions?



RAII: Resource Acquisition Is Initialization

The most important idiom in C++. You already use it everywhere.

- **Acquire** in the constructor, **release** in the destructor
- Object scope == resource lifetime
- Works for memory, files, locks, sockets, GPU buffers

```
std::vector< int > v; // owns a heap buffer
std::unique_ptr< T > p; // owns one T
std::ifstream f{...}; // owns an OS file handle
std::lock_guard lk{m}; // owns a mutex lock
```



Default copy bites: Rule of Three (C++98)

String a; String b = a; → both share the same pointer → **double-free** on destruction. If you touch any one, write all three.

```
String(const String& rhs);           // deep copy
String& operator=(const String& rhs); // deep copy
~String();                          // delete[]
```



But copying is expensive...

A String holding a megabyte. The source dies right after the call, why copy?

```
String make_greeting();           // returns a big string
String s = make_greeting();       // copy? allocate + memcpy?
```



What if there was a way...

to transfer ownership of the heap buffer instead of duplicating it?



Move semantics



A small little lie

C++ has a function called `std::move`. It doesn't move.

- It is a **static_cast** to an rvalue reference
- It changes the **value category**, not the data
- It signals: *“this object is expiring, take from it”*



An analogy: Spirited Away

In Studio Ghibli's *Spirited Away*, Chihiro's parents are transformed by the witch Yubaba.

- They still **exist**: they are not destroyed
- But their **identity is gone**: taken from them
- Chihiro herself loses her **name**, becoming “Sen”



Move in action

```
struct Person {
    std::string parents;
    std::string past_memories;
};

int main() {
    Person chihiro{
        "Father Akio Ogino and Mother Yuko Ogino",
        "Happy memories with friends and a sad goodbye"
    };

    // The cast that signals expiry, Yubaba takes them:
    Person taken = std::move(chihiro);

    // chihiro still exists. But what is in her now?
}
```



What is left of Chihiro?

The moved-from object is **valid but unspecified**: the “nameless” state.

- The destructor will still run safely
- We can **assign** new contents to it
- But reading the current contents? The standard says nothing.

```
std::println("{} ", chihiro.parents); // probably "", but don't rely on it
```



Reviving the Nameless

A moved-from object isn't dead, it's an **empty vessel**. Assign to it, and it's whole again.

```
// chihiro remembers, with Haku's help:  
chihiro.parents = "Father Akio Ogino and Mother Yuko Ogino";  
chihiro.past_memories = "Remembers Haku, and her own name";  
  
std::println("{} ", chihiro.parents); // OK, defined value
```



Note 1: Two kinds of move

In programming languages, there are two flavors of “move”:

- **Destructive move**: source object disappears immediately after the move.
- **Non-destructive move**: source object still exists, lives until end of scope.

C++ chose the **non-destructive** one (it fits the existing lifetime rules).

 The price: you can still *touch* a moved-from object. Its state becomes a real question. (See Note 9.)



Note 2: Move is nothing special

Just another **function overload**. The **&&** overload lets us steal from temporaries.

```
void Fun(std::vector< int >& byRef);           // lvalue ref
void Fun(const std::vector< int >& byConstRef); // const lvalue ref
void Fun(std::vector< int >&& byRvalueRef);    // rvalue ref ← new

std::vector v{2, 3, 4};
Fun(v);                                     // byRef
Fun({3, 5, 6}); // byRvalueRef ← temporary picks the &&
Fun(std::move(v)); //byRvalueRef
```




Note 3: Move vs. copy

Two apartments analogy: buy all-new furniture vs. hire a moving van.
Copy is $O(n)$ + a heap allocation. Move is $O(1)$, two pointer writes.

```
void Copy(char** dst, char** src, size_t n) {  
    *dst = new char[n];    // allocate  
    memcpy(*dst, *src, n); // copy bytes  
}  
  
void Move(char** dst, char** src) {  
    *dst = *src;           // swap pointer  
    *src = nullptr;        // source is empty  
}
```



Note 4: Move is a partial swap

The move “steals” the pointers. The source is left empty but **destroyable**.

```
class String {
    size_t      mLen{};
    std::unique_ptr< char[] > mData{};
public:
    // copy: deep, allocates
    String(const String& rhs)      { *this = rhs; }
    String& operator=(const String& rhs);

    // move: steals pointers
    String(String&& rhs) noexcept
    : mLen { std::exchange(rhs.mLen, 0) }
    , mData{ std::exchange(rhs.mData, nullptr) }
    {}
    String& operator=(String&& rhs) noexcept;
};
```



Note 5: std::move doesn't move (recap)

A static_cast to &&. No data is touched. The compiler then picks the matching && overload, *that's* where the transfer happens.

```
template< typename T >
auto&& move(T&& arg) {
    return static_cast< std::remove_reference_t< T >&& >(arg);
}
```



Note 6: Where did lvalue come from?

1962, CPL (precursor to BCPL $\rightarrow$ B $\rightarrow$ C $\rightarrow$ C++).

“ *An expression evaluable in LH mode produces an address.
The assignment command assigns the RH value to the LH
address.* ”



Note 6: Where did lvalue come from?

- **lvalue**: has an address and a name (something you can take the address of)
- **rvalue**: a temporary, the result of an expression, a literal

```
int i = 5;           // i is lvalue, 5 is rvalue
int j = i;          // j is lvalue, i is lvalue 0_0
i = compute();      // compute() returns an rvalue
```



Note 6: C++11 needs more categories

With move semantics, lvalues can be *moved*. New categories:

- **lvalue**: has identity, not movable
- **xvalue** (“expiring”): has identity, *is* movable (`std::move(x)`)
- **rvalue**: pure rvalue, a temporary literal or return-by-value

```
expression
 /      \
 glvalue  rvalue
 /  \    /  \
 lvalue xvalue prvalue
```



Note 6: In code

Rule of thumb: if it has a name you can spell, it's an lvalue. Otherwise, some kind of rvalue.

```
Object base{};
Object o2 = base;
Object o3 = GetValue();
Object o4 = std::move(base);
Object o5 = GetOtherValue();
- Object o6 = 5;
```

// lvalue (like o2, o3, o4, o5, o6)
// base = lvalue (also a glvalue)
// Object&& GetValue(); xvalue (also a glvalue)
// xvalue (also a glvalue)
// Object GetOtherValue(); prvalue
// prvalue



Note 7: && parameters are NOT const

We *want* to modify the source, that is the whole point. Strip the const.

```
String(const String& rhs); // OK: we read rhs, we don't modify it
String(String&& rhs);     // NOT const: we are about to gut it!
```




Note 8: Move only pays off with dynamic memory

A class with only stack-bound data, three floats, moves as fast as it copies. There is nothing to “steal” .

```
struct Vec3 { float x, y, z; }; // 12 bytes, no heap
```



Note 8: Where move actually helps

Move shines when the class owns a pointer to a **separately-allocated**

resource:

- `std::string`, `std::vector`, `std::unique_ptr`
- file handles, sockets, GPU buffers
- entries in a custom memory pool

No such resource? Don't bother writing custom moves.



Note 9: The moved-from object

After `std::move`, the source is in a “**valid but unspecified**” state. Valid means: the destructor still runs safely, and you can still **assign** to it.

```
String src{"Hello"};
String other{ std::move(src) };
std::println("{}" , src.c_str()); // ← what is in src? unspecified
```



Note 9: Two safe rules

- **Simple rule:** never touch a moved-from object again.
- **Expert rule:** you may reuse it, but **first** bring it back to a known state by assigning to it.

When in doubt, treat C++ as if it had destructive moves. “Use after move” is a code smell.



Note 10: Never `std::move` a return value

The compiler can build `result` directly in the caller's slot, no copy, no move, free. That's **RVO** (Return Value Optimization).

```
std::string Fun() {  
    std::string result{};  
    result = "Hello"s;  
    return std::move(result);    // ← DON'T  
}
```



Note 10: Why `std::move` on return hurts

- `std::move` on a return **disables RVO**
- You force a move where you would have had nothing
- The only exception: ref-qualified member functions (Note 20)



Note 11: Forward the move to your base class

Once you write your own move op, the compiler doesn't forward to the base. **You** must.

```
struct Fruit {  
    Fruit& operator=(Fruit&&) { puts("Base"); return *this; }  
};  
  
struct Apple : Fruit {  
    Apple& operator=(Apple&& rhs) {  
        // BUG: Fruit's move-assign is never called  
        puts("Derived");  
        return *this;  
    }  
};
```



Note 11: The fix

`std::move(rhs)` is safe here: only the `Fruit` subobject is passed to the base. The `Apple` part stays untouched.

```
Apple& operator=(Apple& rhs) {  
    Fruit::operator=(std::move(rhs)); // ← forward the base part  
    puts("Derived");  
    return *this;  
}
```




Note 12: Never `std::move` a temporary

C++17 mandates copy-elision: `Lifeguard{}` is built **directly** into `obj`, zero ctor calls. `std::move` kills the elision and forces a move you didn't need. **Trust the compiler.**

```
Lifeguard obj{ Lifeguard{} };           // OK: copy-elided
Lifeguard obj{ std::move(Lifeguard{}) }; // BAD: kills the elision
```



Note 13: The forwarding reference

A reference written T&& is **not always** an rvalue reference. It is a **forwarding reference** if and only if:

1. The form is exactly T&& (or auto&&)
2. **Type deduction** is required

```
Apple&& a1 = Fun();           // rvalue ref (no deduction)
auto&& a2 = Fun();            // forwarding ref
void Fun(Apple&& f);          // rvalue ref
template< typename T > void Fun(T&& f); // forwarding ref
void Fun(auto&& f);           // forwarding ref
```



Note 14: std::forward

A forwarding reference binds to both lvalues and rvalues. Blindly calling `std::move` would steal from lvalues, disaster.

```
template< typename T >
auto make_apple(T&& value) {
    return Apple{ std::move(value) }; // BAD: steals from lvalues
}

auto str = "long string"s;
Apple x = make_apple(str); // str is now empty!
```



Note 14: The right tool

`std::forward< T >` casts to rvalue **only** if `T` was deduced as an rvalue.

Rule of thumb: deduced `T&&` $\rightarrow$ `std::forward`; plain `T&&` $\rightarrow$

`std::move`.

```
template< typename T >
auto make_apple(T&& value) {
    return Apple{ std::forward< T >(value) }; // preserves category
}
```



Note 15: STL containers want noexcept

The vector resizes. It needs the **strong exception guarantee**. If your move can throw, it falls back to **copy**.

```
struct Lifeguard {  
    Lifeguard(Lifeguard&&) { puts("move"); } // NOT noexcept  
};  
  
std::vector< Lifeguard > v;  
v.push_back(Lifeguard{});  
v.push_back(Lifeguard{}); // → "copy ctor" (!)
```



Note 15: Mark it noexcept

Now the vector uses your move. **Always mark moves noexcept.**

```
Lifeguard(Lifeguard&&) noexcept { puts("move"); }  
Lifeguard& operator=(Lifeguard&&) noexcept { return *this; }
```



Note 16: The Rule of Five

Once you write **any** of these, the compiler stops generating others. Track them carefully.

The five special members:

- destructor
 - copy constructor
 - copy assignment
 - ~~move constructor~~
- SCIE** INSTITUTE **move assignment**



Note 16: The Rule of Five

Modern C++: declare all five, or **none of them** (Rule of Zero, let RAII members do the work).

Godbolt link: <https://godbolt.org/z/x3c51MvEr>



Note 16: What the compiler still gives you

Touch one → lose others. The compiler stops guessing.

You declare ↓	copy ctor	copy assign	move ctor	move assign
Nothing	✓	✓	✓	✓
Destructor	✓ (deprecated)	✓ (deprecated)	X	X
Copy ctor	-	✓	X	X
Move ctor	deleted	deleted	-	X



Note 17: Even = default counts

A virtual destructor (even defaulted) **disables** the implicit move ops. `std::is_move_constructible_v` is misleading, copy is the fallback for move.

```
class Base {  
public:  
    virtual ~Base() = default;    // ← looks harmless  
};  
  
Base b{};  
Base b2 = std::move(b);          // copy ctor, NOT move ctor
```



Note 17: Be explicit

Write one, write all five. Rule of Five in action.

```
class Base {  
public:  
    virtual ~Base()  
    Base(const Base&)  
    Base& operator=(const Base&)  
    Base(Base&&) noexcept // ← restore  
    Base& operator=(Base&&) noexcept  
};
```



Note 17: Core Guideline C.67

A polymorphic class should suppress public copy/move.

```
class Base {  
public:  
    virtual ~Base() = default;  
    virtual std::unique_ptr< Base > clone() const = 0; // ← explicit copy  
  
protected:  
    Base(const Base&) = default; // ← not public: no slicing  
    Base& operator=(const Base&) = default;  
    Base(Base&&) noexcept = default;  
    Base& operator=(Base&&) noexcept = default;  
};
```



Note 17: BAD EXAMPLE - Slicing in action

```
struct Base {  
    virtual void who() const { std::println("Base"); }  
};  
  
struct Derived : Base {  
    int extra = 42;  
    void who() const override { std::println("Derived"); }  
};  
  
void take(Base b) { b.who(); } // ← by value: SLICES  
  
Derived d;  
take(d);  
Base copy = d;  
// prints "Base", not "Derived"!  
// copy is a plain Base, extra lost
```



Note 18: std::initializer_list hides copies

The `{...}` forwards to `std::vector's initializer_list` constructor. Its elements are **const** → they must be **copied** out, not moved.

```
class Apple {
public:
    template< typename... T >
    Apple(T&&... t) : mV{ std::forward< T >(t)... } {} // ← innocent?
private:
    std::vector< std::string > mV;
};

Apple{ LongStr(), LongStr(), LongStr() }; // 7 allocations (!)
```



Note 18: The workaround

Build the vector ourselves with a fold expression. 4 allocations instead of

7. Watch for braced containers whenever you forward.

```
template< typename... Ts >
auto forward_push_back(Ts&&... ts) {
    using T = std::remove_cvref_t< decltype((ts, ...)) >;
    std::vector< T > v;
    v.reserve(sizeof...(Ts));
    (v.push_back(std::forward< Ts >(ts)), ...);
    return v;
}
```



Note 18: Wiring it into Apple

The constructor forwards into `forward_push_back` instead of straight into the `{...}`, the `initializer_list` is bypassed, every element is moved.

```
class Apple {  
public:  
    template< typename... T >  
    Apple(T&&... t)  
        : mv{ forward_push_back(std::forward< T >(t)...) } // ← no braces trap  
    {}  
private:  
    std::vector< std::string > mv{};  
};  
  
Apple{ LongStr(), LongStr(), LongStr() }; // 4 allocations
```




Note 19: `std::move_if_noexcept`

Writing a generic container? You don't know if T's move is `noexcept`.
`move_if_noexcept` silently falls back to **copy** if the move could throw.
Internal use only.

```
template< typename T >
class AwesomeContainer {
    T data{};
public:
    AwesomeContainer(AwesomeContainer&& rhs) noexcept
    : data{ std::move_if_noexcept(rhs.data) } // ← safe fallback
    {}
};
```



Note 20: Ref-qualifiers

`String{"Hello"}.append(" , world!")` calls `append` on a temporary. The returned reference points to a soon-dead object → we **copy** out. What if `append` could tell whether `*this` is a temporary? (Godbolt link: <https://godbolt.org/z/noErnrnTq>)

```
class String {  
public:  
    String& append(const char* s);    // works for any *this  
};  
  
String s = String{"Hello"}.append(" , world!");    // append on a temporary!
```



Note 20: & and && after the function

- `& append(...)`: only callable on lvalue objects
- `&& append(...)`: only callable on rvalues, returns the guts

This is the rare *legitimate* `std::move` on a return, the compiler can't see this optimization.

```
class String {  
public:  
    String& append(const char* s) & { Concat(s); return *this; }  
    String&& append(const char* s) && { Concat(s); return std::move(*this); }  
};
```



Note 20: With vs. without ref-qualifiers

Same call on a temporary: `String{"Hello"}.append(" world!")`.

	Without ref-qualifiers	With & / && overloads
Overload picked	<code>String& append(...)</code>	<code>String&& append(...), rvalue *this</code>
Return value	<code>String&</code> to a dying temporary	<code>String&&</code> , the temporary's guts
Initializing s	copy constructor runs	move constructor runs
Cost	new allocation + element copy	pointer steal, $O(1)$



Note 20: With vs. without ref-qualifiers

Same call on a temporary: `String{"Hello"}.append(" , world!")`.

```
String s = String{"Hello"}.append(" , world!");  
// without ref-qualifiers → copy ctor  
// with ref-qualifiers → move ctor
```



Note 20: C++23: one function with “deducing this”

The `& / &&` pair duplicates the body. C++23 lets `*this` be an explicit, deduced parameter, write the logic once. `self` deduces to `String&` for lvalues and `String&&` for rvalues; `std::forward` returns the matching category.

```
class String {  
public:  
    template < typename Self >  
    auto&& append(this Self&& self, const char* s) {  
        self.Concat(s);  
        return std::forward< Self >(self); // keeps the lvalue/rvalue category  
    }  
};
```



Note 21: Ref-qualify your assignment

`entry_at` was refactored to return by value. The assignment still compiles, and silently writes to a temporary. Almost always a bug.

```
Storage entry_at(size_t pos); // returns by VALUE (refactor)
db.entry_at(2) = Storage{5}; // compiles. silently assigns to a temporary!
```



Note 21: Block assignment-to-temporary

Lvalue-qualify (&) the assignment ops. Now `db.entry_at(2) = ...` is a compile error if `entry_at` returns by value. Touch one, declare all five.

```
struct Storage {  
    Storage()  
    Storage(const Storage&)  
    Storage(Storage&&)  
    Storage& operator=(const Storage&) &  
    Storage& operator=(Storage&&) &  
    = default;  
    = default;  
    = default;  
    = default; // ← &  
    = default; // ← &  
};
```




Note 22: user-declared vs user-defined

Three nested categories. A special member is **trivial** *only if not user-provided*.

- **user-declared**: you wrote a declaration (even = default or = delete)
- **user-provided**: you declared without = default/= delete
- **user-defined**: you wrote a body

 user-defined $\subset$ user-provided $\subset$ user-declared



Note 22: {} is user-provided

An empty body counts as user-provided. The destructor is **non-trivial**, `std::optional< Apple >` emits a real destructor call.

```
class Apple { public: ~Apple() {} };           // user-provided
std::optional< Apple > a{};
```



Note 22: = default keeps it trivial

= default is user-declared but **not** user-provided. The compiler emits no destructor code. Prefer = default over {}. Prefer no declaration at all (Rule of Zero).

```
class Apple { public: ~Apple() = default; }; // user-declared, NOT provided  
std::optional< Apple > a{};
```



Note 23: Implicit move on return (named local)

When the local's type differs from the function's return type, RVO can't apply, but the compiler inserts an implicit `std::move` for you.

```
struct Derived : Lifeguard {};  
Lifeguard Fun() {  
    Derived obj{};  
    return obj;    // implicit move (RVO can't apply: type differs)  
}
```



Note 23: Implicit move on return (by-value parameter)

By-value parameters are excluded from RVO, but C++ still inserts an implicit move on a plain `return val;`. You don't need `std::move`.

```
Lifeguard Fun(Lifeguard val) {  
    return val;    // implicit move  
}
```



Note 23: C++20 fixed the last gap

Before C++20, returning a named rvalue-ref parameter produced a **copy**. C++20 added rvalue-ref params to the “move-eligible” list. Modern compiler → free move; older → an explicit `std::move` here is fine.

```
Lifeguard Fun(Lifeguard&& val) {  
    return val;           // pre-C++20: copy. C++20+: implicit move.  
}
```



Note 23 - In practice

<https://godbolt.org/z/8dnrbsMEs>



Note 24: Move semantics checklist

- Mark move ctor & move assign **noexcept**
- Leave the source **destroyable** (and assignable)
- Protect against **self-move** in operator=(**&&**)
- Move **all** members in your move op (future-proof)
- **std::exchange** or **swap** pointer members: **std::move** on a raw pointer does nothing
- Either don't touch a moved-from object, or assign to it first



Note 24: Checklist (continued)

- **Don't** `std::move` a return value (let RVO happen), exception: `ref-qualified` `&&` members
- Move only helps if there's a heap (or pool) allocation under the hood
- Provide STL-like classes that handle moves for users
- **Never move from a `const`**, and rvalues are never `const`
- **Never move from an lvalue** you'll use again
- Be **stingy** with `std::move`, temporaries don't need it
- Remember: the fallback for move is copy



Rule of Zero: the goal

If your members are already RAll, declare **nothing**. The compiler gives you all five, correct, noexcept when possible. **Rule of Five only when Rule of Zero won't do.**

```
class Player {  
    std::string      name_;  
    std::vector< int >  scores_;  
    std::unique_ptr< Sprite > sprite_;  
};
```





References

- Andreas Fertig, *Notebook C++: About Move Semantics* (2024)
- Siyu (Alice) Peng, *Lightning Talk: std::move & Spirited Away: When Nameless Objects Walk the Spirited World*, the metaphor used in the intro
- Howard Hinnant, *Move semantics introduction* (the original proposals)
- Scott Meyers, *Effective Modern C++*, items 23-30
- [cppreference: value categories](#)
- [cppreference: move constructors](#)



Conclusion

- **RAII** ties resources to scope. Always.
- **Move semantics** is just an overload, &&, that lets temporaries hand over their guts.
- `std::move` is a cast. It does nothing at runtime.
- **Rule of Zero** by default. **Rule of Five** when you can't.
- **noexcept your moves**, or the STL falls back to copy.

Now we know what && really means.